

GraphMind: Framework for Efficient Exploration of LLM Agent Topologies with Graph Neural Networks

This project is available on our [github](https://github.com/vitolev/GraphMind). <https://github.com/vitolev/GraphMind> if the link does not work.

Project Overview

This project aims to efficiently explore the space of multi-agent LLM system topologies using Graph Neural Networks (GNNs) as a surrogate model. Instead of exhaustively testing all possible agent configuration graphs through expensive LLM evaluations, we leverage a trained GNN to predict performance, significantly reducing computational costs while maintaining exploration effectiveness.

Application Domain: Mathematical problem solving using multi-agent reasoning systems.

Core Challenge: Determining which graph topology of agent interactions leads to optimal performance without prohibitively expensive LLM evaluations.

Solution: Iterative pipeline that:

1. Generates candidate graph topologies
2. Uses a GNN surrogate to predict performance
3. Selectively evaluates top candidates with LLMs
4. Retrains the GNN model with new data
5. Repeats to progressively refine topology search

Current State of the Project (Project Milestone)

In the following section, we will present current parts of the pipeline with current development status. Most part of the pipeline currently have some simple algorithms that are placeholders for advanced algorithms that we will implement in the following month until the end of the course. There will also be a recap of development status in Development Status section.

Pipeline Architecture

The framework operates through an iterative 6-step pipeline that progressively refines understanding of optimal multi-agent graph topologies while minimizing expensive LLM evaluations. Current main pipeline is in [pipeline/main_pipeline.py](#).

Step 1: Graph Generation

Purpose: Generate candidate multi-agent topologies for the current iteration.

Location: [graph_generation/graph_generation.py](#), function [generate_graph_batch\(\)](#)

What it does:

- Generates K candidate graph structures from the configuration space

- Initially uses random sampling (current state)
 - Can transition to informed strategies (similarity-based generation from top performers)
 - Returns GraphSet with Graph objects containing node types and edge connections
 - Outputs generation metrics (duration, count, etc.)
-

Step 2: GNN Prediction

Purpose: Predict performance of all generated graphs without costly LLM evaluations.

Location: `gnn_models/model_manager.py`, function `predict_batch_performance()`

What it does:

- Converts GraphSet to PyG HeteroData format for GNN input
 - Runs forward pass through trained GNN model
 - If model not yet fitted (first iteration), assigns random scores (0.0-0.1)
 - Updates each Graph object with predicted `gnn_score`
 - Outputs prediction metrics (best/worst/mean scores, inference time, RMSE vs actual)
 - **Current state:** Model that we are using currently converts the graphs to a vector with some basic features of the graph (num of nodes, num of edges...) and then performs linear regression on this data.
-

Step 3: Graph Selection

Purpose: Select top-M graphs for LLM evaluation based on GNN predictions and past performance.

Location: `data_management/graph_storage.py`, function `select_top_graphs()`

What it does:

- Merges top-K graphs from current batch into `good_graphs_set`
 - Selects `eval_k_best` top graphs for evaluation
 - Maintains sorted order by combined GNN and historical LLM scores
 - Enforces maximum size constraints on persistent graph collection
 - Returns selected GraphSet ready for LLM evaluation
-

Step 4: LLM Evaluation

Purpose: Empirically evaluate selected graphs using LLM agents on mathematical reasoning tasks.

Location: `evaluation/llm_evaluator.py`, function `evaluate_selected_graphs()`

What it does:

- Evaluates selected graphs on mathematical problem set
- **Current implementation:** Synthetic scoring heuristic (structure-based + random + type bonuses)
 - Base score: `edges / (nodes^2)` - rewards denser graphs
 - Random component: `uniform(0.0, 0.3)`
 - Bonus: +0.2 if graph contains `type_a` nodes

- Updates each Graph with `llm_score` and evaluation time
 - Computes RMSE between GNN predictions and actual LLM evaluations
 - Placeholder ready for actual LLM integration via LangGraph
-

Step 5: Training Data Update

Purpose: Integrate newly evaluated graphs into the training dataset.

Location: `data_management/graph_storage.py`, function `update_training_data()`

What it does:

- Adds evaluated graph-performance pairs to training dataset
 - Manages training dataset growth and composition
 - Outputs metrics on new samples added and total dataset size
-

Step 6: GNN Retraining

Purpose: Retrain GNN model on expanded training dataset to improve predictions.

Location: `gnn_models/model_manager.py`, function `retrain_gnn_model()`

What it does:

- Retrains GNN model on all accumulated graph-performance pairs
 - Improves model's ability to predict topology performance
 - Outputs retraining metrics (loss, training time, etc.)
-

Metrics and Analysis

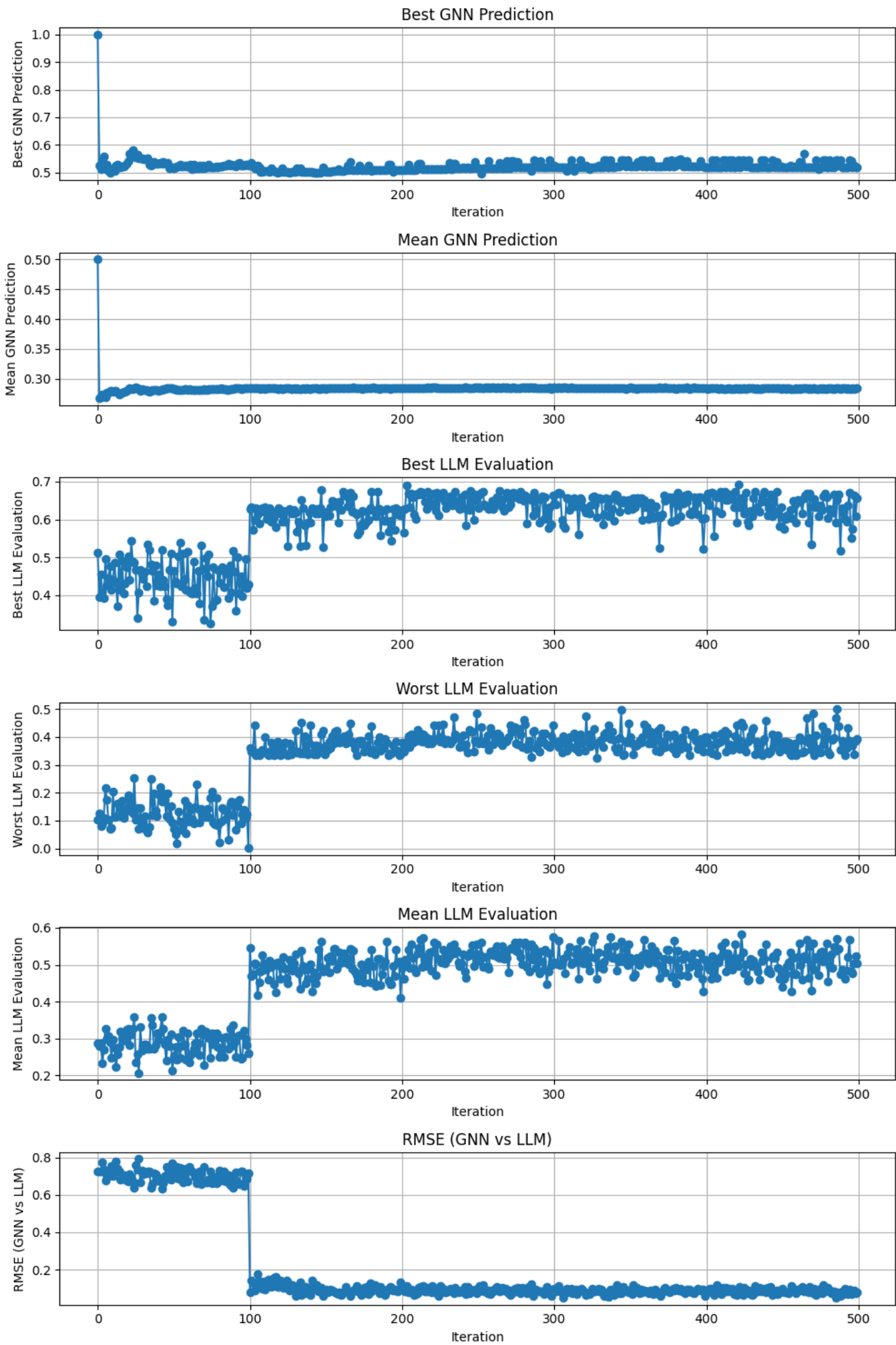
All steps output metrics dictionaries containing:

- Step name and duration
- Number of samples processed
- Key statistics (best/worst/mean values)
- Step-specific metadata

Location: `post_processing/metrics_aggregator.py`

- `flatten_metrics_dict()` - converts nested metrics to flat structure
- `create_metrics_dataframe()` - creates DataFrame from all iterations
- `compute_iteration_summary()` - computes aggregate statistics

All of the data from the loops is saved in DataFrame and can be analyzed. After first experiments, we already got some of the basic visualizations:



How to Run

Prerequisites

```
pip install -r requirements.txt
```

Configuration

Edit `config/experiment_config.yaml` to set pipeline parameters:

```
# Core pipeline settings
max_iterations: 50
num_graphs_per_iteration: 100

# Graph generation
generation_strategy: "random" # or "similar_to_training"
min_nodes: 5
max_nodes: 20
node_types: ["type_a", "type_b", "type_c"]

# Selection parameters
top_k_to_keep: 500 # Keep best K graphs in memory
eval_k_best: 10 # Evaluate top K each iteration

# Model parameters
gnn_architecture: "hetgat" # "gat", "hetgat", "gcn", "graphsage"
retrain_frequency: 2 # Retrain every N iterations

# Output
experiment_name: "exp_001"
```

Run Pipeline

```
python main.py
```

This will:

1. Load configuration from `config/experiment_config.yaml`
2. Set up logging to `logs/{experiment_name}.log`
3. Initialize GNN model and load/create training data
4. Execute iterative pipeline (Steps 1-6 repeated for `max_iterations`)
5. Generate metrics dataframe
6. Run post-processing analysis
7. Save results to `logs/{experiment_name}/analytics/`

Key Files Reference

File	Purpose
main.py	Entry point, sets up logging and calls pipeline
pipeline/main_loop.py	Core iteration loop, orchestrates 6 steps
graph_generation/graph_generation.py	Step 1: Generate candidate graphs
gnn_models/model_manager.py	Step 2 & 6: Prediction and retraining
evaluation/llm_evaluator.py	Step 4: Graph evaluation
data_management/graph_storage.py	Data persistence and selection logic
post_processing/metrics_aggregator.py	Metrics collection and analysis
post_processing/analytics.py	Post-run visualization and reporting
config/settings.py	Configuration schema and defaults

Development Status

This is a **Project Milestone** implementation with:

- Complete 6-step pipeline architecture
- Metrics collection and aggregation framework
- Modular step functions with clear interfaces
- Synthetic evaluation heuristic (ready for real LLM integration)
- Post-processing visualization
- Advanced generation strategies (random implemented, similarity-based pending)

Next Steps

1. Implement actual LLM evaluation via LangGraph in Step 4
2. Add advanced graph generation strategies (similarity-based)
3. Implement visualization and reporting in post-processing
4. Tune GNN model architecture selection
5. Validate core hypotheses with real evaluations